

API_CONTRACT

HelixLLM HTTP API Contract

Source-verified from code 2026-07-06. Every claim below cites a real `file:line` in `submodules/helix_llm` and is drawn from the `route/handler/struct` code — not from memory, not guessed (§11.4.6).

- **Module:** `github.com/HelixDevelopment/HelixLLM (go.mod:1)`, Go 1.26.1 (`go.mod:3`)
- **Repo HEAD at verification:** `e035599d4664dcc781b5f7e54f13220d3aee7993 (e035599 govern(cascade): backfill 11.4.142-165 anchor band)`
- **Compiles today:** YES — `go build ./... exits 0` (evidence: `docs/research/07.2026/00_master/evidence/helixllm_contract/go_build.txt`)
- **Route enumeration evidence:** `docs/research/07.2026/00_master/evidence/helixllm_contract/route_enumeration_grep.txt`

1. Listen address, port, TLS, transports

The binary entry point is `cmd/helixllm/main.go`. The HTTP server is built at `cmd/helixllm/main.go:151-158`:

```
srv := server.New(server.Options{
    Host:      cfg.Server.Host,
    Port:      cfg.Server.Port,
    TLSCert:    cfg.Server.TLSCert,
    TLSKey:     cfg.Server.TLSKey,
    ...
})
```

Config defaults (`internal/shared/config/config.go:38-41`):

Field	Env var	Default
Host	HELIX_HOST	0.0.0.0
Port	HELIX_PORT	8443
TLSCert	HELIX_TLS_CERT	./certs/cert.pem
TLSKey	HELIX_TLS_KEY	./certs/key.pem

TLS is MANDATORY for serving. ListenAndServe returns an error immediately if either cert or key is empty (internal/server/server.go:87-89). The server listens with **TLS 1.3 minimum** (server.go:100) and serves **both HTTP/3 (QUIC over UDP)** (server.go:105-110) **and HTTP/2 (TLS over TCP)** (server.go:113-117) on the same host:port. ALPN = h3, h2, http/1.1 (server.go:101); an Alt-Svc: h3=":<port>" header is added to every response (server.go:196-198).

CONFIRMED: default base URL is https://0.0.0.0:8443. The challenge runner's default -base-url flag is https://localhost:8443 (cmd/helixllm/main.go:40) — matches the inventory note.

UseLlamaCpp toggle — UNCONFIRMED as named; the real toggle is LlamaServerEmbed

grep -rn "UseLlamaCpp" over the whole repo returns **nothing** — there is no field/env named UseLlamaCpp. The actual embedded-llama.cpp toggle is:

- LlamaServerEmbed bool env:"HELIX_LLAMA_SERVER_EMBEDDED" default:"true" (config.go:76)
- When true, main.go:208-247 starts an embedded brain.LlamaServer on LlamaServerPort (HELIX_LLAMA_SERVER_PORT, default 8080, config.go:75) and overrides LocalRPCHost=127.0.0.1, LocalRPCPort=LlamaServerPort (main.go:245-246).
- The Brain's llama.cpp URL is http://<LocalRPCHost>:<LocalRPCPort> (main.go:269), default localhost:50052 (config.go:48-49) unless the embedded override fires. The local model default is Llama-3.1-70B-Instruct-Q4_K_M (config.go:47).

2. Complete registered-route inventory (verbatim registrations)

Routes are attached in main.go in this order: base server routes (in server.New), then gateway.RegisterRoutes (main.go:370), knowledge.RegisterKnowledgeRoutes (main.go:381), agents.RegisterAgentRoutesWithExtras (main.go:468), control.RegisterRoutes (main.go:470).

Method	Path	Registration (file:line)	Handler	Auth?
GET	/internal/health		healthHandler	none

Method	Path	Registration (file:line)	Handler	Auth?
		internal/ server/ server.go:167		
GET	/internal/ metrics	server.go:173 /:175	Prometheus	none
GET	/metrics	internal/ gateway/ router.go:58	Prometheus (outside /v1 auth)	none
POST	/v1/chat/ completions	router.go:72	HandleChatCompletions	API- key
POST	/v1/ completions	router.go:73	HandleCompletions	API- key
GET	/v1/models	router.go:74	HandleListModels	API- key
GET	/v1/ models/:id	router.go:75	HandleGetModel	API- key
POST	/v1/ embeddings	router.go:76	HandleEmbeddings	API- key
GET	/v1/ hardware	router.go:79	inline (returns HardwareProfile)	API- key
POST	/v1/ messages	router.go:84	HandleMessages (Anthropic)	API- key
GET	/ws	router.go:87	HandleWebSocket (outside /v1 auth)	none ²
POST	/internal/ knowledge/ ingest	internal/ knowledge/ api.go:28	handleIngest	API- key ³
POST	/internal/ knowledge/ query	knowledge/ api.go:29	handleQuery	API- key ³
GET	/internal/ knowledge/ collections	knowledge/ api.go:30	handleCollections	API- key ³
GET	/internal/ knowledge/ stats	knowledge/ api.go:31	handleStats	API- key ³

Method	Path	Registration (file:line)	Handler	Auth?
POST	/v1/agents/chat	internal/agents/api.go:113	agentChatHandler	API-key ³
GET	/v1/agents/tools	agents/api.go:114	agentToolsHandler	API-key ³
POST	/v1/agents/tools/execute	agents/api.go:115	toolExecuteHandler	API-key ³
POST	/v1/agents/coordinate	agents/api.go:116	coordinateHandler	API-key ³
POST	/v1/agents/plan	agents/api.go:117	planHandler	API-key ³
POST	/v1/agents/memory/remember	agents/api.go:118	memoryRememberHandler	API-key ³
POST	/v1/agents/memory/recall	agents/api.go:119	memoryRecallHandler	API-key ³
GET	/v1/cache/stats	agents/api.go:124	cacheStatsHandler	API-key ³
GET	/internal/cluster/status	internal/control/api.go:121	handleStatus	API-key ³
POST	/internal/cluster/probe	control/api.go:122	handleProbe	API-key ³
POST	/internal/cluster/deploy	control/api.go:123	handleDeploy	API-key ³
POST	/internal/cluster/rebalance	control/api.go:124	handleRebalance	API-key ³

¹ **Auth finding (RESOLVED — DZ-05, 2026-07-07):** The gateway API-key middleware was originally applied only to the gateway's own /v1 group (router.go:63-64: v1 := r.Group("/v1"); v1.Use(gwmw.APIKeyAuth(...))). The agent routes (r.Group("/v1/

agents")), the sibling cache-stats route (`r.Group("/v1")`), and the `/internal/cluster/* + /internal/knowledge/*` groups are registered on **separate** RouterGroup instances that do NOT inherit the gateway group's middleware, so they were **unauthenticated**. DZ-05 remediation: `RegisterRoutes / RegisterKnowledgeRoutes / RegisterAgentRoutes(WithExtras)` now accept an optional `authMW ...gin.HandlerFunc` applied to their group(s) via `.Use()`; `main.go` wires the SAME `gwmw.APIKeyAuth(cfg.Auth.APIKeys)` middleware (`main.go:387-388,475,477`) — identical semantics to `/v1` (empty `HELIX_AUTH_API_KEYS` $\Rightarrow$ open-access; keys configured $\Rightarrow$ enforced). Proven `RED` $\rightarrow$ `GREEN`: `unauth` $\Rightarrow$ **401**, valid Authorization: `Bearer <key>` $\Rightarrow$ **200** (guards `internal/{control,knowledge,agents}/auth_dz05_test.go`, evidence docs/qa/dz05_endpoint_auth_20260707/).

² **/ws (open by design decision — DESIGN QUESTION, not fixed)**: `/ws` runs the Brain over a WebSocket. `gwmw.APIKeyAuth` reads the `Authorization: Bearer` header, which browser-native WebSocket clients cannot set (only subprotocols / query params). Gating `/ws` with the header-based middleware would break browser clients — a **client-contract change** that cannot be decided from evidence alone (§11.4.6). Left unchanged; requires an operator decision on the WS credential channel (header vs `?api_key=` query param vs subprotocol) before it can be authenticated.

³ **Enforced only when HELIX_AUTH_API_KEYS is non-empty** — identical to the gateway `/v1` group. `/internal/health`, `/internal/metrics`, and `/metrics` remain **intentionally public** (liveness probe + Prometheus scraper convention; the `/metrics` comment at `router.go:56` documents the intent) and are NOT gated.

3. Authentication & headers

- **Scheme**: Bearer token — `Authorization: Bearer <token>` (`internal/gateway/middleware/auth.go:35-41`).
- **Open-access mode**: when `APIKeys` (env `HELIX_AUTH_API_KEYS`, passed as `cfg.Auth.APIKeys`, `main.go:371`) is the empty string, the middleware calls `c.Next()` and allows every request (`auth.go:29-32`). Keys are a comma-separated list; each token is trimmed and compared (`auth.go:44-50`).
- **401 body** on failure is OpenAI-error JSON `{"error":{"message":..., "type":"invalid_request_error"}}` (`auth.go:58-64`).
- **Request Content-Type**: `application/json` (all handlers use `gin.ShouldBindJSON`, e.g. `openai.go:49`).
- Additional `/v1` middleware (`router.go:65-68`): `RateLimit` (per-minute per-IP; `cfg.Server.RatePerMinute`), `SecurityHeaders`, and — only when

`cfg.Features.TOON` is true — ContentNegotiation (TOON content negotiation).

4. OpenAI-compatible endpoints

The wire structs live in `pkg/api/openai.go` and are the actual types the gateway handlers bind/emit (`internal/gateway/openai.go:20` imports `pkg/api`).

4.1 POST /v1/chat/completions

- Request struct: `api.ChatCompletionRequest` (`pkg/api/openai.go:4-18`) — fields `model`, `messages[] {role, content(string|[]ContentPart), name, tool_calls, tool_call_id}`, `temperature`, `top_p`, `n`, `stream`, `stop`, `max_tokens`, `presence_penalty`, `frequency_penalty`, `user`, `tools[]`, `tool_choice`.
- Bound at `internal/gateway/openai.go:48-49`. Empty `model` defaults to `llama-3.1-70b` (`openai.go:60-63`).
- Response struct: `api.ChatCompletionResponse` (`pkg/api/openai.go:62-69`) — `id`, `object: "chat.completion"`, `created`, `model`, `choices[] {index, message, finish_reason}`, `usage {prompt_tokens, completion_tokens, total_tokens}`. Built by `internalToOpenAI` (`openai.go:931-968`), returned at `openai.go:365`.
- **Streaming (SSE)**: when `stream: true` and no tools, chunks stream as `api.ChatCompletionChunk` (`object: "chat.completion.chunk"`, `pkg/api/openai.go:78-96`) via the OpenAI SSE writer (`openai.go:259-303`). Content-Type `text/event-stream`, Cache-Control: `no-cache`, Connection: `keep-alive`, X-Accel-Buffering: `no` (`internal/gateway/streaming.go:32-38`); each event is `data: {json}\n\n` (`streaming.go:54`) and the stream terminates with `data: [DONE]\n\n` (`streaming.go:61`).
- **Behavior notes (not shape, but load-bearing for test banks)**: when a Brain is wired the handler heavily rewrites the request — replaces the caller's system prompt with an internal one (`openai.go:109-137`), injects a respond tool + sets `tool_choice` (`openai.go:140-166`), truncates/strips oversized & XML-injected messages (`openai.go:76-101, 189-209`), applies the RAG hook (`openai.go:238-240`), and **forces non-streaming when tools are present** (`openai.go:254-257`), re-emitting the full response as a single SSE chunk if the client asked for streaming (`openai.go:330-363`). The wire shapes stay OpenAI-compatible.
- **No-Brain dev fallback**: returns a canned greeting (`openai.go:376-396` non-stream; `streamChatCompletions` `openai.go:401-434`).

Example request (derived from `ChatCompletionRequest`):

POST /v1/chat/completions

Authorization: Bearer <key>

Content-Type: application/json

```
{"model": "llama-3.1-70b", "messages": [{"role": "user", "content": "What  
is 2+2?"}], "stream": false}
```

Example response (derived from ChatCompletionResponse):

```
{"id": "chatcmpl-  
helix-1a2b3c4d", "object": "chat.completion", "created": 1751760000,  
"model": "llama-3.1-70b",  
"choices": [{"index": 0, "message":  
{"role": "assistant", "content": "4"}, {"finish_reason": "stop"}],  
"usage":  
{"prompt_tokens": 10, "completion_tokens": 6, "total_tokens": 16}}
```

Example SSE stream (text/event-stream):

```
data: {"id": "chatcmpl-helix-...", "object": "chat.completion.chunk", "created"
```

```
data: [DONE]
```

4.2 POST /v1/completions

- Request: `api.CompletionRequest` (pkg/api/openai.go:118-124) — `model`, `prompt` (string), `max_tokens`, `temperature`, `stream`. Bound at `openai.go:440-441`; the prompt is wrapped into a single user message (`openai.go:458-463`).
- Response: `api.CompletionResponse` (pkg/api/openai.go:126-133) — `object: "text_completion"`, `choices[]`{`text`, `index`, `finish_reason`}, `usage`. Returned at `openai.go:481-498`. **No SSE path implemented** — the stream flag is accepted but the handler always returns a single JSON body.

Example:

```
{"model": "llama-3.1-70b", "prompt": "Hello", "max_tokens": 16} →  
{"id": "cmpl-  
helix-...", "object": "text_completion", "model": "...", "choices":  
[{"text": "...", "index": 0, "finish_reason": "stop"}], "usage": {...}}
```

4.3 GET /v1/models and GET /v1/models/:id

- Response: `api.ModelList` {`object: "list"`, `data: []api.Model`} (pkg/api/openai.go:104-115). `Model` = {`id`, `object: "model"`, `created`, `owned_by`}.

- With a Brain wired, `/v1/models` returns `b.Models()` (`openai.go:526-540`); `/v1/models/:id` searches Brain models and 404s with an OpenAI-error body if not found (`openai.go:544-578`). Without a Brain a built-in 3-model list is used (`openai.go:33-37`: `llama-3.1-70b`, `gpt-4o`, `claude-sonnet-4-20250514`).
- `ModelBrain` (used only by these two endpoints) is `brainSvc` (`main.go:374`), aligning with `CONST-036/037` (models come from the provider layer, not hardcoded when a Brain is present).

Example: `GET /v1/models` → `{"object":"list","data":[{"id":"...","object":"model","created":1700000000,"owned_by":"helix"}]}`

4.4 POST /v1/embeddings

- Request: `api.EmbeddingRequest {model, input(string|[]string)}` (`pkg/api/openai.go:142-145`), bound at `openai.go:586-587`. Empty `model` → `text-embedding-ada-002` (`openai.go:598-601`).
- Response: `api.EmbeddingResponse {object:"list", data:[{object:"embedding", embedding:[]float64, index}], model, usage}` (`pkg/api/openai.go:147-158`).
- Delegates to the knowledge-layer `Embedder` when present (`openai.go:604-637`); otherwise returns a zero vector of dim 1536 (or `embedder.Dimension()`) (`openai.go:641-661`) so the endpoint is always reachable.

5. Anthropic-compatible endpoint — POST /v1/messages

Structs in `pkg/api/anthropic.go`.

- Request: `api.MessageRequest (pkg/api/anthropic.go:4-15)` — `model`, `messages[] {role, content(string|[]ContentBlock)}`, `max_tokens (int)`, `system`, `temperature`, `top_p`, `stream`, `stop_sequences[]`, `tools[]`, `tool_choice`. Bound at `internal/gateway/anthropic.go:53-54`. Empty `model` → `claude-sonnet-4-20250514` (`anthropic.go:65-68`). `system` is prepended as a system message in `anthropicToInternal` (`anthropic.go:286-291`).
- Response: `api.MessageResponse (pkg/api/anthropic.go:39-48)` — `id`, `type:"message"`, `role:"assistant"`, `content:[]ContentBlock`, `model`, `stop_reason (end_turn|max_tokens|stop_sequence|tool_use)`, `stop_sequence`, `usage {input_tokens, output_tokens}`. Built by `internalToAnthropic` (`anthropic.go:317-343`), returned at `anthropic.go:102`.

- **Streaming (SSE), Anthropic named-event format**

(anthropic.go:131-205): Content-Type text/event-stream

(anthropic.go:30-36), each event is event: <type>\ndata:

<json>\n\n (anthropic.go:44). Event sequence: message_start →

content_block_start → content_block_delta (text_delta) ... →

content_block_stop → message_delta (carries stop_reason + usage) →

message_stop, then a trailing event: ping (anthropic.go:203). **No**

[DONE] sentinel (distinct from the OpenAI SSE path).

Example request:

POST /v1/messages

```
{"model": "claude-sonnet-4-20250514", "max_tokens": 128,  
  "messages": [{"role": "user", "content": "Hi"}], "stream": false}
```

Example response:

```
{"id": "msg-helix-...", "type": "message", "role": "assistant",  
  "content": [{"type": "text", "text": "Hello!"}], "model": "claude-  
    sonnet-4-20250514",  
  "stop_reason": "end_turn", "stop_sequence": null,  
  "usage": {"input_tokens": 10, "output_tokens": 6}}
```

6. Health & metrics

GET /internal/health (server.go:167, 179-188)

- Returns the health.Report (= observability/pkg/health.Report)
JSON: {status, components: [{name, status, message?, duration, last_checked}], timestamp} (submodules/observability/pkg/health/health.go:32-53).
- **Status codes:** 200 when Status == StatusHealthy, else 503
(server.go:182-186).

GET /internal/metrics and GET /metrics

- Prometheus exposition text (server.go:173/175, router.go:58). /
metrics is registered by the gateway outside the /v1 auth group so scrapers
need no key (router.go:56-58).
-

7. WebSocket — GET /ws

- Upgraded via gorilla/websocket, CheckOrigin allows all origins (internal/gateway/websocket.go:18-20,30).
 - **Speaks the internal type, NOT the OpenAI shape:** the client sends JSON-encoded types.InternalChatRequest frames and receives types.InternalChatResponse (or {"error":...} / {"message":"ok (no brain)"} frames (websocket.go:49-68).
 - types.InternalChatRequest (pkg/types/types.go:48-57): model, messages[] {role, content, name, tool_calls, tool_call_id}, max_tokens, temperature, stream, provider, tools, tool_choice.
 - types.InternalChatResponse (pkg/types/types.go:60-67): id, model, message, finish_reason, usage{prompt_tokens, completion_tokens, total_tokens}, provider.
 - Each frame is answered with a single (non-streaming) b. Complete call (websocket.go:59).
-

8. Auxiliary internal endpoints (request/response shapes)

Agents (internal/agents/api.go)

- POST /v1/agents/chat — req AgentChatRequest{session_id?, messages[], model?} (api.go:13-23) → AgentChatResponse{session_id, response(InternalChatResponse)} (api.go:26-29).
- GET /v1/agents/tools — lists registered agent tools (agentToolsHandler).
- POST /v1/agents/tools/execute — req ToolExecuteRequest{tool(required), params} (api.go:79-82) → ToolExecuteResponse{result, error?} (api.go:85-88).
- POST /v1/agents/coordinate — req CoordinateRequest{task} (api.go:32-34) → CoordinateResponse{tasks[], final_output} (api.go:37-40).
- POST /v1/agents/plan — req PlanRequest{goal} (api.go:43-45).
- POST /v1/agents/memory/remember — req MemoryRememberRequest{session_id, content, importance} (api.go:48-52).
- POST /v1/agents/memory/recall — req MemoryRecallRequest{session_id, query, top_k?} (api.go:55-59) → MemoryRecallResponse{memories[]} (api.go:62-64).
- GET /v1/cache/stats — KV-cache stats (cacheStatsHandler).

- Nil deps (coordinator/planner/memMgr) degrade gracefully to 501/degraded (api.go:90-92).

Knowledge (internal/knowledge/api.go)

- POST /internal/knowledge/ingest — IngestRequest → IngestResult (api.go:15,31-53).
- POST /internal/knowledge/query — QueryRequest → QueryResult (api.go:16,55-77).
- GET /internal/knowledge/collections — []Collection (api.go:79-88).
- GET /internal/knowledge/stats — Stats (api.go:90-99).
- Error body: {"error":{"message":..., "type":"knowledge_error"}} (api.go:105-112).

Control / cluster (internal/control/api.go)

- GET /internal/cluster/status — ClusterStatus{checked_at, healthy, deployments} (api.go:113,119-129).
- POST /internal/cluster/probe — handleProbe (api.go:114).
- POST /internal/cluster/deploy — handleDeploy (api.go:115).
- POST /internal/cluster/rebalance — handleRebalance (api.go:116).

9. UNCONFIRMED / gaps

- **UseLlamaCpp** — no such named field/env exists anywhere in the tree (grep -rn UseLlamaCpp → empty). Mapped to LlamaServerEmbed (HELIX_LLAMA_SERVER_EMBEDDED, default true, config.go:76). See §1.
- **Agent / knowledge / control / /ws / /metrics endpoints are unauthenticated** under the present wiring (only the gateway /v1 LLM group gets APIKeyAuth). This is a real observation from router.go:63-64 vs agents/api.go:102,112 / control/api.go:112 / knowledge/api.go:20 — flagged for the gateway/HelixQA-bank designers, not a documented intent.
- Full field lists for IngestRequest/QueryRequest/Stats and the handleProbe/Deploy/Rebalance request bodies were **not exhaustively read** here (only their registrations + top-level flow); treat their inner shapes as UNCONFIRMED pending a read of internal/knowledge/*.go and internal/control/api.go handler bodies.

Sources verified

All citations are `file:line` in `submodules/helix_llm` @ HEAD `e035599`, cross-checked against the captured route-enumeration `grep` and the `go build ./...` (exit 0) output under `docs/research/07.2026/00_master/evidence/helixllm_contract/`.